

Computer Programming 143 – Lecture 12

Functions IV

Faculty of Engineering



Copyright

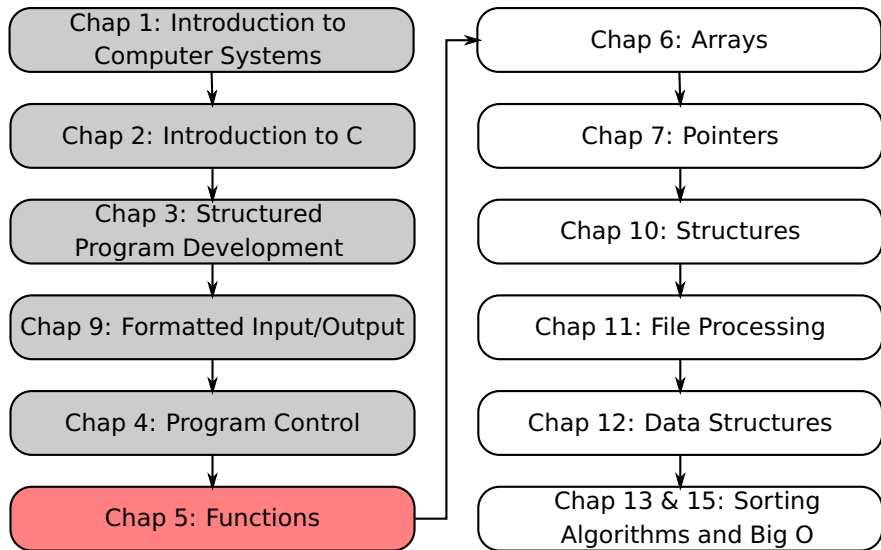
Copyright © 2026 Stellenbosch University
All rights reserved

Disclaimer

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

Module Overview



Lecture Overview

- 1 Recursion (5.14)
- 2 Example: Fibonacci Series (5.15)
- 3 Recursion vs. Iteration (5.16)

5.14 Recursion I

Function calls so far

- Functions call one another in a disciplined and hierarchical manner
- Functions only call other functions – not themselves

Definition of recursion

A **recursive function** is a function that calls itself, either directly or indirectly through another function

5.14 Recursion II

Recursive problem solving

- Function can only solve simplest case(s) directly (**base case(s)**):
 - Function called with base case: returns result of base case
- Function called with more complex case:
 - Breaks problem into two pieces: one it “knows how to do” and one it “does not know how to do”
 - The second part must be a simpler or smaller version of the original problem
 - To solve the second part, the function calls itself (**recursion step**)
 - Function stays “open” while recursion step executes

5.14 Recursion III

Recursion analogy



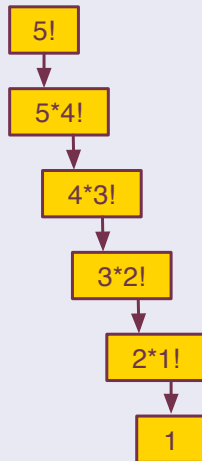
5.14 Recursion IV

Factorial 5!

$$\begin{aligned}5! &= 5 \times 4 \times 3 \times 2 \times 1 \\&= 5 \times (4 \times 3 \times 2 \times 1) \\&= 5 \times (4!)\end{aligned}$$

Similarly,

$$\begin{aligned}4! &= 4 \times (3!) \\3! &= 3 \times (2!) \\2! &= 2 \times (1!) \\1! &= 1\end{aligned}$$



5.14 Recursion V

Factorial $n!$

$$n! = \begin{cases} 1, & n = 1 \\ n \times (n-1)!, & n > 1 \end{cases}$$

Problem

- Write a function to calculate the factorial of a number recursively.

5.14 Recursion VI

Pseudocode

function: factorial of integer n

if n is less than or equal to 1

set the variable to return to 1

else calculate the factorial of $(n-1)$, i.e. call factorial $(n-1)$

set the variable to return to n multiplied by the factorial of $(n-1)$

5.14 Recursion VII

Factorial: recursive implementation

```
// fig05_09.c
// Recursive factorial function.
#include <stdio.h>

long factorial(long n);

int main(void) {
    // calculate factorial(i) and display result
    for (long i = 1; i <= 7; ++i) {
        printf("%ld! = %ld\n", i, factorial(i));
    }
    return 0;
}
```

5.14 Recursion VIII

Factorial: recursive implementation (cont...)

```
long factorial( long n ) {  
    long x;  
    // base case  
    if ( n <= 1 ) {  
        x = 1;  
    }  
    else { // recursive step  
        x = n * factorial( n - 1 );  
    }  
    return x;  
} // end function factorial
```

5.14 Recursion IX

Output

```
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
```

5.15 Example: Fibonacci Series

Fibonacci series

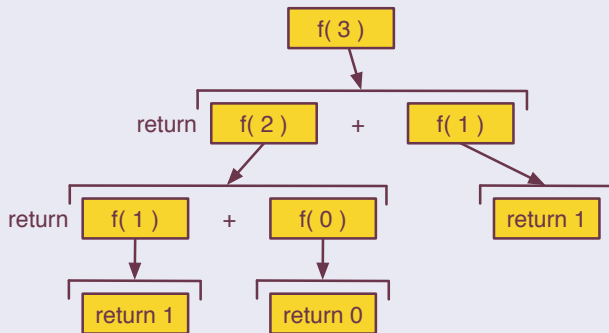
The Fibonacci series, 0, 1, 1, 2, 3, 5, 8, 13, 21, ..., begins with 0 and 1 and has the property that each subsequent Fibonacci number is the sum of the previous two Fibonacci numbers. Any Fibonacci number can therefore be

calculated as

$$\text{Fibonacci}(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ \text{Fibonacci}(n - 1) + \text{Fibonacci}(n - 2), & n > 1 \end{cases}$$

5.15 Example: Fibonacci Series

Fibonacci series (cont...)



5.15 Example: Fibonacci Series

Problem

- Write a function to calculate the Fibonacci series recursively.

Pseudocode

function: fibonacci of integer n

if n is 0 or 1

set return variable to n

else

calculate the fibonacci of $(n-1)$, i.e. call fibonacci $(n-1)$

calculate the fibonacci of $(n-2)$, i.e. call fibonacci $(n-2)$

set return variable to the sum of fibonacci $(n-1)$ and fibonacci $(n-2)$

5.15 Example: Fibonacci Series

Fibonacci series: recursive implementation

```
// fig05_10.c
// Recursive fibonacci function.
#include <stdio.h>
long int fibonacci(long int n); // function prototype
int main(void) {
    // calculate and display fibonacci(number) for 0-10
    for (long number = 0; number <= 10; number++) {
        printf("Fibonacci(%ld) = %ld\n", number,
            fibonacci(number));
    }
    printf("Fibonacci(20) = %ld\n", fibonacci(20));
    printf("Fibonacci(30) = %ld\n", fibonacci(30));
    printf("Fibonacci(40) = %ld\n", fibonacci(40));
    return 0;
}
```

5.15 Example: Fibonacci Series

Fibonacci series: recursive implementation (cont...)

```
long int fibonacci( long int n ) {  
    long int x;  
    // base case  
    if ( n == 0 || n == 1 ) {  
        x = n;  
    } // end if  
    else { // recursive step  
        x = fibonacci( n - 1 ) + fibonacci( n - 2 );  
    } // end else  
    return x;  
} // end function fibonacci
```

5.15 Example: Fibonacci Series

Output

```
Fibonacci(0) = 0
Fibonacci(1) = 1
Fibonacci(2) = 1
Fibonacci(3) = 2
Fibonacci(4) = 3
Fibonacci(5) = 5
Fibonacci(6) = 8
Fibonacci(7) = 13
Fibonacci(8) = 21
Fibonacci(9) = 34
Fibonacci(10) = 55
Fibonacci(20) = 6765
Fibonacci(30) = 832040
Fibonacci(40) = 102334155
```

5.16 Recursion vs. Iteration I

Recursion vs. Iteration

- All problems that can be solved with recursion can also be solved with iteration
- Recursion is more processor and memory intensive
- Sometimes the recursion solutions are more elegant

Today

Functions IV

- Definition of recursion
- Example: Fibonacci series
- Recursion vs. iteration

Next lecture

Arrays I

- Introduction to, definition of and use of arrays

Homework

- 1 Study Sections 5.14-5.16 in Deitel & Deitel
- 2 Do Self Review Exercises 5.1(k)-(q) in Deitel & Deitel
- 3 Do Exercises 5.34, 5.36 in Deitel & Deitel